

Week 5

Task 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week5
saichaitanya@GeethzzSai:~/Practical_week5$ nano 1.c
```

```
GNU nano 8.7.1 1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

int main()
{
    int pipefd[2];
    pid_t pid;

    char buffer[100];
    int i;

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
    {
        // Child - Consumer
        close(pipefd[1]);
```

```
// Child - Consumer
close(pipefd[1]);

printf("Consumer (Child) started. PID = %d\n", getpid());

while (read(pipefd[0], buffer, sizeof(buffer)) > 0)
{
    printf("Consumer received: %s", buffer);
}

close(pipefd[0]);

printf("Consumer finished.\n");

exit(0);
}
else
{
    // Parent - Producer
    close(pipefd[0]);

    printf("Producer (Parent) started. PID = %d\n", getpid());

    for (i = 1; i <= 10; i++)
    {
        snprintf(buffer, sizeof(buffer),
                 "Data %d\n", i);

        write(pipefd[1], buffer, strlen(buffer));
    }

    close(pipefd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);
}
```

```

close(pipefd[1]);

wait(NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

printf("Producer finished.\n");
printf("Communication time: %.6f seconds\n", elapsed);

if (elapsed > 0)
{
    printf("Communication efficiency: %.2f messages/second\n",
        10.0 / elapsed);
}

return 0;
}

```

COMPILE AND RUN:

```

saichaitanya@GeethzzSai: ~/f × + ∨
saichaitanya@GeethzzSai:~$ cd Practical_week5
saichaitanya@GeethzzSai:~/Practical_week5$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week5$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week5$ ./1

```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/F × + ▾
saichaitanya@GeethzzSai:~$ cd Practical_week5
saichaitanya@GeethzzSai:~/Practical_week5$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week5$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week5$ ./1
Producer (Parent) started. PID = 665
Consumer (Child) started. PID = 666
Consumer received: Data 1
Data 2
Data 3
Data 4
Data 5
Data 6
Data 7
Data 8
Data 9
Data 10
Consumer finished.
Producer finished.
Communication time: 0.000910 seconds
Communication efficiency: 10985.65 messages/second
saichaitanya@GeethzzSai:~/Practical_week5$ |
```

Task 2: Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

CODE:

```
saichaitanya@GeethzzSai: ~/f  ×  +  ▾
GNU nano 8.7.1 2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    // Create first child
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0)
    {
        // First child: execute ls -l

        close(pipefd[0]);

        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);
    }
}
```

```
if (pid1 == 0)
{
    // First child: execute ls -l

    close(pipefd[0]);

    // Redirect stdout to pipe
    dup2(pipefd[1], STDOUT_FILENO);

    close(pipefd[1]);

    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls failed");
    exit(1);
}

// Create second child
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    exit(1);
}

if (pid2 == 0)
{
    // Second child: execute grep ".c"

    close(pipefd[1]);

    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);
}
```

```
if (pid2 < 0)
{
    perror("fork");
    exit(1);
}

if (pid2 == 0)
{
    // Second child: execute grep ".c"

    close(pipefd[1]);

    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep failed");
    exit(1);
}

// Parent closes both pipe ends
close(pipefd[0]);
close(pipefd[1]);

// Wait for both children
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/Practical_week5$  
saichaitanya@GeethzzSai:~/Practical_week5$ nano 2.c  
saichaitanya@GeethzzSai:~/Practical_week5$ gcc 2.c -o 2  
saichaitanya@GeethzzSai:~/Practical_week5$ ./2  
1.c  
2.c  
3.c  
Pipeline execution completed.  
saichaitanya@GeethzzSai:~/Practical_week5$ |
```